

Correction — Exercice 13 : Responsive mobile-first

Solution — Lecture du CSS de départ

Le CSS fourni fonctionne « out of the box ». À retenir :

- **Reset minimal** : `box-sizing: border-box` + reset margin sur body. Indispensable avant tout responsive.
- **Mobile par défaut** : `flex-direction: column` sur `.cards`. Toutes les cards empilées.
- **Tablette (≥ 768px)** : passage en row + wrap. Le `flex: 1 1 calc(50% - 0.5rem)` = grow=1, shrink=1, base=50% moins la moitié du gap → 2 colonnes pile.
- **Desktop (≥ 1024px)** : `flex: 1 1 calc(25% - 0.75rem)` → 4 colonnes. Le `0.75rem` = 3/4 du gap (3 intervalles entre 4 items).

Vérification dans DevTools :

- À 374px : 1 colonne (mobile).
- À 768px pile : 2 colonnes apparaissent. Tester aussi 767px (encore 1 colonne).
- À 1024px : 4 colonnes.

Solution — Navbar mobile-first

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Navbar responsive</title>
  <style>
    * { box-sizing: border-box; }
    body { margin: 0; font-family: system-ui, sans-serif; }

    /* ≡ MOBILE par défaut (< 768px) ≡ */
    .navbar {
      display: flex;
      justify-content: space-between;
      align-items: center;
      padding: 1rem;
      background: #1f2937;
      color: white;
    }
    .navbar-logo {
      color: white;
      text-decoration: none;
      font-weight: bold;
      font-size: 1.25rem;
    }
    .navbar-menu {
      display: none; /* caché sur mobile */
      list-style: none;
      margin: 0;
      padding: 0;
    }
    .navbar-burger {
      display: block; /* visible sur mobile */
      background: transparent;
      color: white;
      border: 1px solid white;
      padding: 0.4rem 0.6rem;
      cursor: pointer;
      font-size: 1.25rem;
    }
    .navbar-menu a {
      color: white;
      text-decoration: none;
      padding: 0.5rem 1rem;
      display: block;
    }

    /* ≡ TABLETTE (≥ 768px) ≡ */
    @media (min-width: 768px) {
```

```
.navbar-burger {
  display: none;
}

.navbar-menu {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 0.25rem 1rem;
}
}

/* === DESKTOP (≥ 1024px) === */
@media (min-width: 1024px) {
  .navbar-menu {
    display: flex;
    gap: 0;
  }
}

/* === XL (≥ 1440px) - bonus === */
@media (min-width: 1440px) {
  .navbar-logo {
    font-size: 1.5rem;
  }
}
</style>
</head>
<body>
  <nav class="navbar">
    <a class="navbar-logo" href="#">MaMarque</a>
    <ul class="navbar-menu">
      <li><a href="#">Accueil</a></li>
      <li><a href="#">Produits</a></li>
      <li><a href="#">À propos</a></li>
      <li><a href="#">Contact</a></li>
    </ul>
    <button class="navbar-burger" aria-label="Menu">≡</button>
  </nav>
</body>
</html>
```

Points clés :

- **Mobile d'abord** : `display: none` sur le menu, `display: block` sur le burger. Logique « par défaut, c'est mobile ».
- **Cascade additive** : chaque media query **ajoute** ou **modifie** seulement les règles changées. On ne réécrit jamais le bloc entier.
- `grid-template-columns: 1fr 1fr` en tablette pour faire 2 colonnes : élégant, pas besoin de calcul de pourcentage.
- **Desktop** : on revient en `flex` parce que l'horizontal pur est plus simple qu'une grid 1 ligne.

Pourquoi `min-width` plutôt que `max-width` ? Avec `min-width`, le CSS pour mobile reste simple et **lisible sans condition**. C'est l'ordre naturel de la pensée : « voici mon site mobile, et pour les écrans plus grands je rajoute X et Y ».

Solution — Container queries

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <title>Container queries démo</title>
  <style>
    body { margin: 0; font-family: system-ui, sans-serif; padding: 2rem; display: flex; gap: 2rem; }

    .sidebar {
      width: 300px;
      container-type: inline-size;
      background: #f3f4f6;
      padding: 1rem;
    }

    .hero {
      flex: 1;
      container-type: inline-size;
      background: #fefce8;
      padding: 1rem;
    }

    /* Layout par défaut – vertical (small container) */
    .card {
      display: flex;
      flex-direction: column;
      gap: 0.75rem;
      background: white;
      border-radius: 8px;
      padding: 1rem;
      box-shadow: 0 1px 3px rgba(0,0,0,0.1);
    }

    .card img {
      width: 100%;
      height: 150px;
      object-fit: cover;
      border-radius: 4px;
    }

    /* Conteneur ≥ 400px → layout horizontal */
    @container (min-width: 400px) {
      .card {
        flex-direction: row;
        align-items: center;
      }

      .card img {
        width: 150px;
      }
    }
  </style>
</html>
```

```

        height: 100px;
        flex-shrink: 0;
    }
}
</style>
</head>
<body>
  <aside class="sidebar">
    <h2>Sidebar 300px</h2>
    <article class="card">
      
      <div>
        <h3>Card en sidebar</h3>
        <p>Layout vertical car le conteneur fait 300px.</p>
      </div>
    </article>
  </aside>

  <main class="hero">
    <h2>Hero (pleine largeur restante)</h2>
    <article class="card">
      
      <div>
        <h3>Card en hero</h3>
        <p>Layout horizontal car le conteneur fait plus de 400px.</p>
      </div>
    </article>
  </main>
</body>
</html>

```

Réponses

1. Différence pratique @media vs @container

- **@media (min-width: 400px)** : réagit à la **largeur de la viewport** (la fenêtre du navigateur). Sur un desktop 1920px, **toutes** les cards passeront en horizontal, même celles dans une sidebar de 300px.
- **@container (min-width: 400px)** : réagit à la **largeur du conteneur parent** qui a **container-type**. Une card dans une sidebar 300px reste verticale, une card dans le main 1200px passe en horizontal — **dans la même page**.

C'est un changement de paradigme : **le composant n'a plus besoin de savoir où il vit**.

2. Support 2026

- ▶ Chrome/Edge 105+ (août 2022).
- ▶ Firefox 110+ (février 2023).
- ▶ Safari 16+ (septembre 2022).

Support universel sur tous les navigateurs modernes (~95 % global selon caniuse).
Acceptable en production en 2026.

3. Cas d'usage où container queries gagnent

Cas type : une page e-commerce avec une grille principale de cards (3 colonnes) + une sidebar « produits similaires » qui affiche les mêmes cards.

Avec `@media` : il faudrait **dupliquer** le composant `.card` en `.card-sidebar` avec un layout vertical. Ou utiliser des modifieurs BEM (`card card--vertical`) — ce qui force le développeur à choisir le bon modifier au bon endroit, fragile.

Avec `@container` : **un seul composant** `.card` , qui s'adapte automatiquement. Réutilisable partout sans modifier. C'est l'objectif final du component-driven design.

Autres cas :

- Widgets embarqués dans des iframes ou des emplacements de taille variable.
- Design systems atomiques.
- Composants tiers (réseaux sociaux, players vidéo) qui doivent s'adapter à l'emplacement où ils sont incrustés.